

Giochi di Numeri

Dall'aritmetica al pensiero ricorsivo

Gionata Massi¹, Alberto Cesaretti² e Pamela Sanchini²

¹ Istituto di Istruzione Superiore “Savoia Benincasa” di Ancona
gionata.massi@savoiaabenincasa.it

² Liceo Scientifico e Musicale “Einstein” di Rimini
{alberto.cesaretti,pamela.sanchini}@liceoeinstein.it ”

Sommario

Questo resoconto descrive un corso extracurricolare di 10 ore, svolto con 15 studenti del primo biennio del Liceo Matematico, per introdurre il pensiero ricorsivo usando l'aritmetica come contesto e *Scheme* come linguaggio. A partire da tre primitive sui naturali, *zero?*, *s* (successore) e *p* (predecessore), e da regole di manipolazione simbolica, gli studenti sono guidati alla definizione ricorsiva di procedure computazionali. Il percorso è coerente con i traguardi e gli obiettivi della proposta CINI per il primo biennio della scuola secondaria di secondo grado. L'approccio ha inizialmente richiesto tempi distesi per la notazione prefissa e la formalizzazione; nelle ultime lezioni ha favorito la capacità di prevedere l'esito dei programmi e di riconoscere la struttura ricorsiva delle soluzioni.

1 Contesto e obiettivi

Il percorso è stato realizzato nell'ambito del progetto dell'IIS “Savoia Benincasa” “Citizen scientists of the future”, finanziato dal Piano Nazionale di Ripresa e Resilienza (PNRR), Missione 4 – Istruzione e Ricerca, Investimento 3.1 “Nuove competenze e nuovi linguaggi”. Il gruppo destinatario era costituito da 15 studenti del primo biennio del Liceo Matematico; non erano richieste competenze pregresse di programmazione, ma soltanto familiarità con i numeri naturali, il significato delle quattro operazioni, l'ordine di precedenza e la valutazione delle espressioni con parentesi.

La scelta del tema è stata concordata tra gli insegnanti di matematica e di informatica e prende ad oggetto di studio un contesto che dovrebbe essere ben chiaro agli studenti, anche se gli autori riscontrano frequentemente nella propria pratica delle difficoltà: molti studenti sanno usare procedure aritmetiche, ma non riescono a descriverle come processi eseguibili da una macchina, e quasi nessuno ricorda la divisione euclidea. L'aritmetica ha quindi funzionato come *contesto* già familiare allo studente, non come fine del percorso, e l'obiettivo centrale era la transizione dal “saper fare” al “saper far fare” aritmetica. I nuclei disciplinari informatici trattati sono stati: descrizione algoritmica, linguaggio formale, strutture dati elementari (atomi, liste), ricorsione, valutazione di espressioni e correttezza delle procedure.

Il percorso è stato progettato per perseguire i seguenti traguardi e obiettivi della Proposta di Indicazioni Nazionali per l'insegnamento dell'Informatica nella Scuola [4]: comprensione della necessità di un esecutore automatico per esprimere algoritmi in modo non ambiguo (T-S-1); riconoscimento della generalità dell'algoritmo rispetto al singolo input (T-S-2); giustificazione della correttezza di un algoritmo (T-S-3); previsione del risultato di un programma senza eseguirlo (O-S-P-2); progettazione di programmi modulari con procedure e funzioni (O-S-P-6); scrittura di semplici programmi in un linguaggio testuale (O-S-P-7).

2 Scelte metodologiche e strumenti

L'ambiente scelto è *WeScheme*¹ perché permette di valutare espressioni in linguaggio *Scheme* direttamente nel browser. Il linguaggio si presta a costruire una *forma mentis* ricorsiva, capace di definire procedure e strutture che si riferiscono a sé stesse, senza richiedere una sintassi complessa. Tre forme sintattiche, *define*, *lambda* e *cond*, sono sufficienti per costruire tutte le procedure proposte. Altri linguaggi avrebbero comportato l'introduzione di concetti aggiuntivi.

Scheme, inoltre, è il linguaggio scelto nei testi da cui gli autori hanno tratto ispirazione [1, 2, 3]. Come in [3], lo schema ricorsivo non è stato introdotto come regola astratta, ma fatto emergere per imitazione e ripetizione: gli studenti hanno osservato numerose procedure già scritte, ne hanno discusso la struttura comune e l'hanno poi riprodotta su problemi nuovi. A livello di progettazione degli algoritmi si è applicato il principio della *ricorsione strutturale* [2]: la struttura della procedura ricalca la struttura induttiva del dato in ingresso. Nel caso dei numeri naturali ciò significa scomporre ogni numero n diverso da zero nel suo predecessore ($p\ n$), riconducendo il problema a un'istanza via via più vicina al caso base *zero*.

Per sostenere la distinzione tra espressione e sua valutazione abbiamo usato sistematicamente quattro mediatori iconici per la stessa espressione (+ 1 2): la scrittura prefissa (Fig. 1a), la struttura a lista (Fig. 1b), l'albero sintattico (Fig. 1c) e il cerchio di valutazione di Bootstrap (Fig. 1d). Il passaggio tra le rappresentazioni (Fig. 1) è stato usato per esplicitare la struttura ricorsiva dell'espressione e l'ordine di valutazione.

Nel caso dei numeri naturali e delle operazioni su di essi sono stati usati i regoli Cuisenaire.

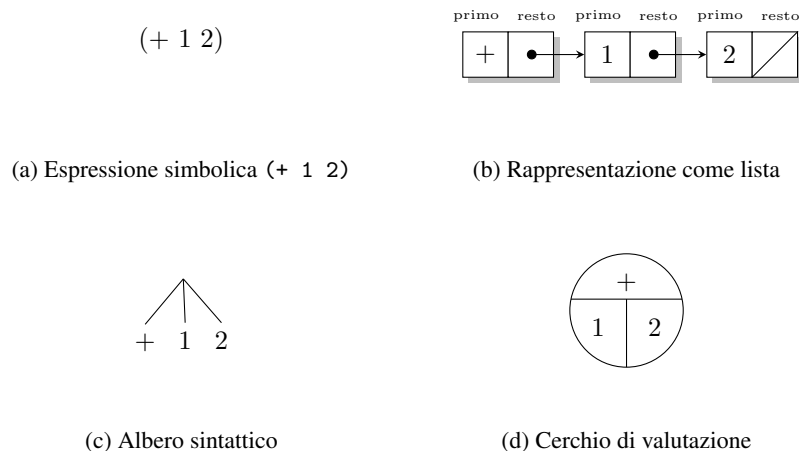


Figura 1: Quattro rappresentazioni dell'espressione (+ 1 2) usate come mediatori

Per ragionare sulle espressioni, rappresentate come liste (Fig. 1b), agli studenti sono state fornite le definizioni in Scheme delle costanti, dei predicati e delle funzioni per la manipolazione di liste usate

¹Disponibile all'indirizzo: <https://www.wescheme.org/>. Dopo il termine del corso, per semplificare la scrittura dei programmi e visualizzare il processo di calcolo, è stato costruito un ambiente didattico online: <https://gionatamassibenincasa.github.io/process-visualizer/>

nelle prime tre lezioni²: la costante `lista-vuota`, i predicati `atomo?`, `lista?`, `lista-vuota?` e `uguale?`, le procedure `primo`, `resto`, `anteponi` e `lista`. Le primitive aritmetiche `s`, `p` e `zero?` sono state introdotte separatamente e restano in uso per l'intero percorso.

3 Articolazione del percorso

Le 10 ore sono state organizzate in cinque incontri da due ore.

Nel primo incontro si è conosciuto l'ambiente WeScheme e si è lavorato sui concetti di atomo, lista, albero sintattico ed espressione, introducendo la notazione prefissa e il modello di valutazione per sostituzione.

Nel secondo incontro si è introdotta la forma condizionale e gli studenti hanno usato predicati e condizionali per distinguere caso base e passo ricorsivo nella manipolazione di liste di atomi e numeri naturali.

Nel terzo incontro sono state costruite procedure ricorsive su liste: riconoscimento di liste di atomi, semplici algoritmi di ricerca e di sostituzione di valori.

Il quarto e il quinto incontro sono stati dedicati esclusivamente alle operazioni sui numeri naturali: alcune procedure sono state sviluppate dall'insegnante, altre costruite dagli studenti a partire dallo schema già noto; altre ancora sono state presentate con nomi non significativi e fatte riconoscere tramite simulazione.

Addizione, sottrazione e predicati di confronto sono stati definiti senza ricorrere a tabelline o alla notazione in colonna, ma a partire dalle sole primitive `s`, `p`, `zero?` e da uno schema ricorsivo comune: la scomposizione strutturale del numero naturale. La Fig. 2 illustra il caso dell'addizione: il secondo argomento è ridotto ricorsivamente fino al caso base, mentre i successori da applicare al primo restano sospesi. Per ogni valutazione abbiamo affiancato tre piani: la manipolazione dei regoli Cuisenaire, la riscrittura delle espressioni intermedie alla lavagna e la relativa rappresentazione grafica (Fig. 3 per (addizione 3 2))³.

La divisione intera è stata introdotta presentando il listato con il nome non significativo `operazione-misteriosa`: gli studenti, osservando la valutazione di `(operazione-misteriosa 12 4)`, dovevano stabilire se l'operazione fosse loro nota e, in caso affermativo, attribuirle il nome corretto. La definizione ricorsiva della procedura `quoziente` (Fig. 8) esprime il quoziente tramite sottrazioni successive: il primo argomento viene progressivamente ridotto fino a un valore minore del secondo, raggiungendo così il caso base, mentre i successori restano sospesi e vengono sommati durante la fase di contrazione, esattamente come nel caso dell'addizione. Come illustrato in Fig. 4 con i regoli Cuisenaire, questo meccanismo rende visibile, passo dopo passo, la costruzione del quoziente.

Si è infine accennato a come la moltiplicazione possa essere definita come addizione ripetuta e l'elevamento a potenza come moltiplicazione ripetuta, cambiando soltanto l'operatore. La generalizzazione del pattern ricorsivo in una funzione di ordine superiore sarebbe andata oltre gli obiettivi del percorso, ma è stata accennata come esempio di astrazione.

²Per comodità d'uso in classe sono stati adottati nomi italiani al posto dei corrispondenti predefiniti in Scheme: `lista-vuota` per `'()`, `atomo?` (non primitiva, definita in termini di `pair?`), `lista?` per `list?`, `lista-vuota?` per `null?`, `uguale?` per `eq?`, `primo` per `car`, `resto` per `cdr`, `anteponi` per `cons`, `lista` per `list`.

³Con l'eccezione di `minore?` e `sottrazione`, che generano un processo iterativo, le procedure presentate generano processi ricorsivi lineari: la valutazione si espande accumulando operazioni differite (`s`) fino al caso base, per poi contrarsi [1, par. 1.2.1]. L'inefficienza del processo non è stata oggetto di discussione, non rientrando tra gli obiettivi del corso.

```

; (addizione n m) produce la somma di n e m
; es.: (addizione 3 0) -> 3
;      (addizione 3 2) -> 5
(define addizione
  (lambda (n m)
    (cond
      [(zero? m) n]
      [else (s (addizione n (p m)))])))
;
; Esempio di valutazione:
; (addizione 3 2)
; -> (s (addizione 3 (p 2)))
; -> (s (addizione 3 1))
; -> (s (s (addizione 3 (p 1))))
; -> (s (s (addizione 3 0)))
; -> (s (s 3))
; -> (s 4)
; -> 5

```

Figura 2: Definizione ricorsiva di addizione con esempio di valutazione di (addizione 3 2)

4 Risultati e limiti

Le considerazioni che seguono derivano dalle attività e dalle discussioni in classe; non è stato possibile raccogliere ex post dati strutturati o prove comparabili.

Diversi studenti hanno iniziato a leggere i programmi come oggetti da interpretare e discutere, non solo da eseguire: era frequente chiedere di prevedere l'esito di una chiamata prima di lanciarla e di riscriverla passo dopo passo, con un miglioramento percepibile rispetto alle prime lezioni. Gli studenti hanno appreso gradualmente il lessico disciplinare: *espressione*, *procedura*, *caso base*, *espressione intermedia*, *argomento*, *parametro*⁴. La ricorsione, inizialmente percepita come artificio, è stata riconosciuta da una parte della classe come tecnica generale di scomposizione dei problemi.

Il percorso ha anche mostrato limiti chiari. La sintassi prefissa di *Scheme* crea una soglia di ingresso più alta rispetto a linguaggi con notazione infissa. Dieci ore sono sufficienti per costruire intuizioni iniziali robuste, ma non per consolidarle in modo uniforme: una parte della classe ha saputo riutilizzare il pattern ricorsivo in compiti nuovi, mentre altri studenti hanno incontrato difficoltà.

Per una futura edizione sarebbe utile introdurre verifiche iniziali e finali per misurare l'evoluzione nella capacità di predire il comportamento dei programmi e di progettare definizioni ricorsive semplici. Sarebbe inoltre opportuno affiancare a *Scheme* una fase conclusiva di trasferimento in un linguaggio testuale più diffuso, per mostrare la trasferibilità delle competenze acquisite.

Nel complesso l'esperienza suggerisce che, anche nel primo biennio, è possibile lavorare su contenuti fortemente disciplinari senza ridurre l'Informatica all'uso di applicazioni. La costruzione ricorsiva dell'aritmetica ha offerto un contesto motivante proprio perché ha reso esplicito che dietro operazioni note esistono rappresentazioni formali e algoritmi. La scelta degli esempi è coerente con il profilo del Liceo Matematico e richiede adattamenti in altri indirizzi.

⁴La distinzione tra *argomento* (il valore passato nella chiamata) e *parametro* (l'identificatore nella definizione della procedura) segue l'uso adottato in [1, 2].

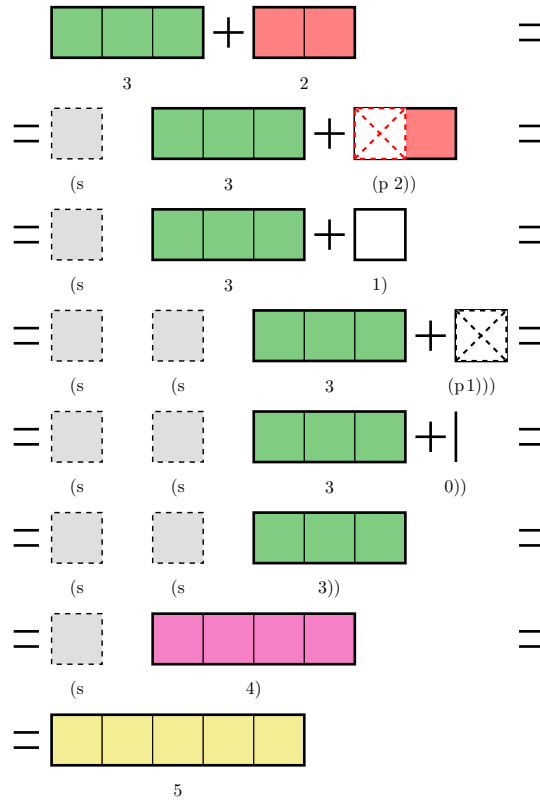


Figura 3: Valutazione di (addizione 3 2) con i regoli Cuisenaire: riduzione del secondo argomento e applicazione dei successori

A Esempi di codice usati nel percorso didattico

Le definizioni delle procedure primitive per l'aritmetica⁵ sono in Fig. 5.

Le procedure ricorsive per esprimere formalmente operatori e predicati matematici e per il calcolo del quoziente della divisione intera sono riportate nelle Figg. 6, 7 e 8.

Riferimenti bibliografici

- [1] Harold Abelson e Gerald Jay Sussman. *Structure and Interpretation of Computer Programs*. Versione *open access*: <https://sarabander.github.io/sicp/>. The MIT Press, 1996, p. 683. ISBN: 9780262510875.
- [2] Matthias Felleisen et al. *How to design programs: An introduction to programming and computing*. 2^a ed. MIT Press, 2018.
- [3] Daniel P Friedman e Matthias Felleisen. *The Little Schemer*. MIT Press, 1995.

⁵s e p sono definite tramite + e - di Scheme per comodità pratica; concettualmente svolgono il ruolo di primitive. La procedura zero? è già fornita dal linguaggio.

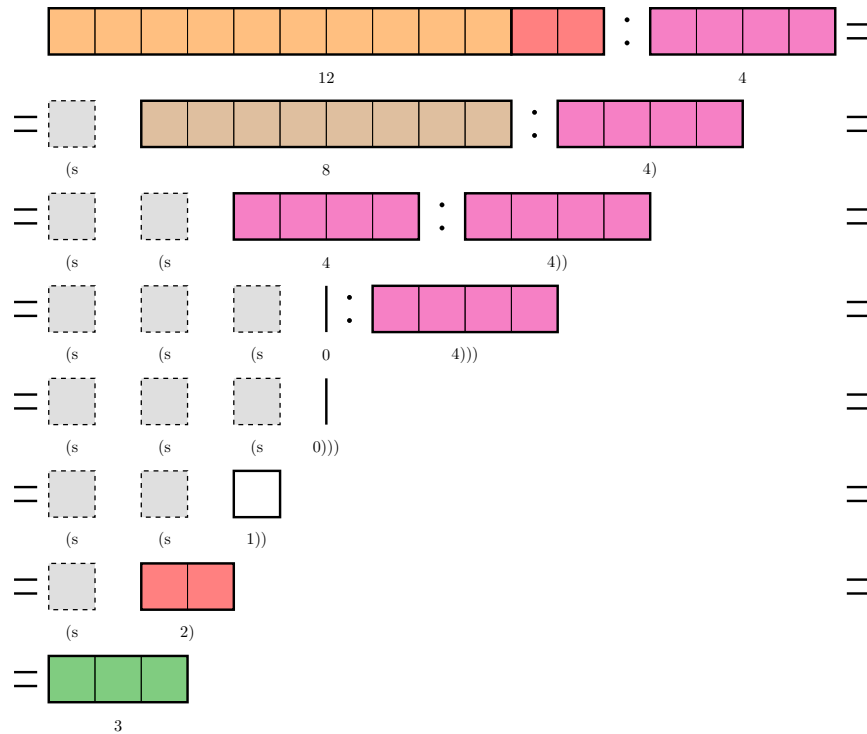


Figura 4: Valutazione di (quoziente 12 4) con i regoli Cuisenaire: sottrazioni ripetute e costruzione del quoziente

```

; (s n) produce il numero che segue immediatamente n
; es.: (s 3) -> 4
(define s
  (lambda (n)
    (+ n 1)))

; (p n) produce il numero che precede immediatamente n
; es.: (p 3) -> 2
(define p
  (lambda (n)
    (- n 1)))

; (zero? n) produce #t se n è zero, #f altrimenti
; es.: (zero? 0) -> #t
;       (zero? 3) -> #f
; è già definito dal linguaggio Scheme

```

Figura 5: Primitive aritmetiche usate per costruire ricorsivamente le altre operazioni

```

; (minore? n m) produce #t se n è minore di m, #f altrimenti
; es.: (minore? 2 5) -> #t
;       (minore? 5 2) -> #f
(define minore?
  (lambda (n m)
    (cond
      [(zero? m) #f]
      [(zero? n) #t]
      [else (minore? (p n) (p m))])))
;
; Esempio di valutazione:
;   (minore? 2 5)
; -> (minore? 1 4)
; -> (minore? 0 3)
; -> #t

```

Figura 6: Definizione ricorsiva di minore? per confronto tra numeri naturali

```

; (sottrazione n m) produce la differenza tra n e m
; es.: (sottrazione 5 2) -> 3
(define sottrazione
  (lambda (n m)
    (cond
      [(zero? m) n]
      [else (sottrazione (p n) (p m))])))
;
; Esempio di valutazione:
;   (sottrazione 5 2)
; -> (sottrazione 4 1)
; -> (sottrazione 3 0)
; -> 3

```

Figura 7: Definizione ricorsiva di sottrazione tramite decremento simultaneo dei due argomenti

- [4] Gruppo di Lavoro CINI “Informatica e Scuola”. *Proposta di Indicazioni Nazionali per l’insegnamento dell’Informatica nella Scuola*. Documento redatto in collaborazione con GII e GRIN. 2017. URL: <https://www.consorzio-cini.it/images/Proposta-Indicazioni-Nazionali-Informatica-Scuola-numerata.pdf>.

```

; (quoziente n m) produce il quoziente della divisione intera di n per m
; es.: (quoziente 12 4) -> 3
(define quoziente
  (lambda (n m)
    (cond
      [(<minore? n m) 0]
      [else (s (quoziente (sottrazione n m) m))])))
;
; Esempio di valutazione:
; (quoziente 12 4)
; -> (s (quoziente 8 4))
; -> (s (s (quoziente 4 4)))
; -> (s (s (s (quoziente 0 4))))
; -> (s (s (s 0)))
; -> (s (s 1))
; -> (s 2)
; -> 3

```

Figura 8: Definizione ricorsiva di quoziente